

Architetture e Sistemi Operativi

Contents

1	Introduzione	3
2	Processore, memoria e istruzioni	4
2.1	Processore	4
2.2	Memoria	4
2.3	Tipi di istruzioni	4
3	Livelli di astrazione dei calcolatori	6
3.1	Principi importanti	6
4	Circuiti Digitali	7
4.1	Sistema binario	7

1 Introduzione

In questo corso andremo a:

- Capire come funziona un elaboratore.
- Capire quali tipi di istruzioni è in grado di eseguire un elaboratore nativamente.
- Come lo si può dotare di un sistema operativo.

Il corso utilizza Windows e Linux come esempi di sistemi operativi.

Codice Teams: 6uk5ntg

Email Docente: marco.delutto@unipi.

2 Processore, memoria e istruzioni

2.1 Processore

Il **processore** è in grado di fare operazioni piccole.

Il processore contiene i **registri**, memorie piccolissime e velocissime utilizzate per memorizzare dati temporanei legati alle istruzioni da eseguire. Ecco alcuni registri speciali:

- **Program Counter (PC)**: un registro speciale che conserva lo stato di esecuzione memorizzando l'indirizzo della prossima istruzione da eseguire.
- **Instruction Register (IR)**: contiene l'istruzione prelevata dalla memoria che deve essere eseguita.
- **Stack Pointer (SP)**: contiene la posizione attuale dello stack.

2.2 Memoria

Il processore è unito ad una **memoria**, costituita da **celle**, è vista come un vettore:

$$M[i] = k$$

Ogni cella ha un valore e la sua capacità è 1 parola (32 o 64 bit).

Una volta che il processore viene acceso, esso esegue:

```
while(true):  
    instr = getInstructionFrom(M)  
    decoded = decode(instr)  
    execute(decoded)  
    determineResults()  
    ...
```

Il processore quindi:

- Estrae istruzioni dalla memoria
- Le decodifica
- Le esegue
- Calcola i risultati

2.3 Tipi di istruzioni

Le istruzioni possono essere:

- Operative
 - Aritmetiche
 - * ADD
 - * SUB
 - * MUL

- * DIV
 - Logiche
 - * AND
 - * OR
 - * NOT
 - Shift / rotazione
 - * SHIFT
 - * ROT
 - ...
- Di Memoria
 - LOAD
 - * Restituisce un valore da un indirizzo:

LOAD val , addr
 - STORE
 - * Salva un valore in un indirizzo:

STORE val , addr
- di Salto
 - "Saltano" pezzi di codice, volendo anche secondo istruzioni, aggiornando il PC alla corrispondente istruzione.

```
inizio :  
  \dots  
  jmp fine :  
  —skipped code—  
fine :
```

3 Livelli di astrazione dei calcolatori

Per capire come opera un computer osserviamo dei livelli di astrazione, sviluppati a "torre" nel seguente modo:

applicazioni
sistema operativo
architettura (set istruzioni)
 μ - architettura (processore)
logica (componenti)
circuiti digitali (0/1, 5V/0V)
circuiti analogici
dispositivi
fisica

Non andremo a vedere niente delle caratteristiche hardware degli ultimi 3 livelli, non ci interessa.

Per capire:

- Noi partiamo col realizzare dei circuiti digitali che servono per creare dei componenti
- Tali componenti ci servono per realizzare un processore
- Al processore dovremo dare in pasto delle istruzioni
- Le istruzioni servono al sistema operativo e alle applicazioni

Tra i livelli di "architettura" e "sistema operativo" si trova un **interfaccia**, genericamente attribuita al livello **i**. Questo significa che:

- Il livello del sistema operativo corrisponde al livello **i** (si trova al livello dell'interfaccia)
- Il livello dell'architettura corrisponde al livello **i - 1**

Al livello **i + 1**, tra sistema operativo e applicazioni, si trovano le **chiamate al sistema operativo**, chiamate **API**.

Ogni livello conosce solo il **livello inferiore** a se, questo è per mantenere i livelli superiori attraverso diversi livelli inferiori; per esempio un telefono Samsung e un Motorola sono realizzati in maniera diversa, ma entrambi utilizzano Android, questo è possibile proprio grazie al fatto che i livelli non si concentrano sui livelli soprastanti.

Un altro esempio: non si possono portare app iOS su Android.

Spesso esistono dei traduttori che permettono di convertire istruzioni (x64,x86) per permettere compatibilità con sistemi operativi e processori differenti.

3.1 Principi importanti

I componenti che svolgono le istruzioni seguono questi concetti:

- **Gerarchia**
- **Modularità**: presenza di moduli per la specializzazione e composizione del lavoro.
- **Regolarità**

Si usano moduli più piccoli per svolgere azioni più grandi quando combinati.

4 Circuiti Digitali

4.1 Sistema binario

Noi utilizziamo il sistema decimale, ovvero un sistema **posizionale**, dove ogni cifra possiede un valore diverso a seconda della sua posizione.

Anche il **sistema binario** è un sistema posizionale, ma le cifre sono solo 2 (0,1).

- **Esempio decimale:**

$$13_{10} = (1 \cdot 10^1) + (3 \cdot 10^0)$$

- **Esempio binario:**

$$10_2 = (1 \cdot 2^1) + (0 \cdot 2^0) = 2_{10}$$

Nei computer si utilizza il sistema binario per via della sua semplicità e della sua compatibilità con i moduli logici, per esempio una moltiplicazione è più facile da svolgere in binario tramite un modulo **AND**, invece che in decimale.

Per rappresentare i numeri negativi in binario ci sono due modi:

- **Modulo e segno:** si riserva il **MSB** (Most Significant Bit, più a sinistra) per il segno (1 se negativo, 0 se positivo).
- **Complemento a 2:** nato perche complicate le operazioni con modulo e segno (if else); si ha un algoritmo per ottenere il binario di un numero negativo:
 1. Si trasforma il numero in binario.
 2. Si fa il NOT di ogni bit
 3. si aggiunge "+1"